

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 13

MONGODB DBMS



MONGODB DBMS: OUTLINE

MongoDB Data Model and Database Design

MongoDB Query Language (MQL)

Schema Validation

Aggregation Framework

DATA MODEL

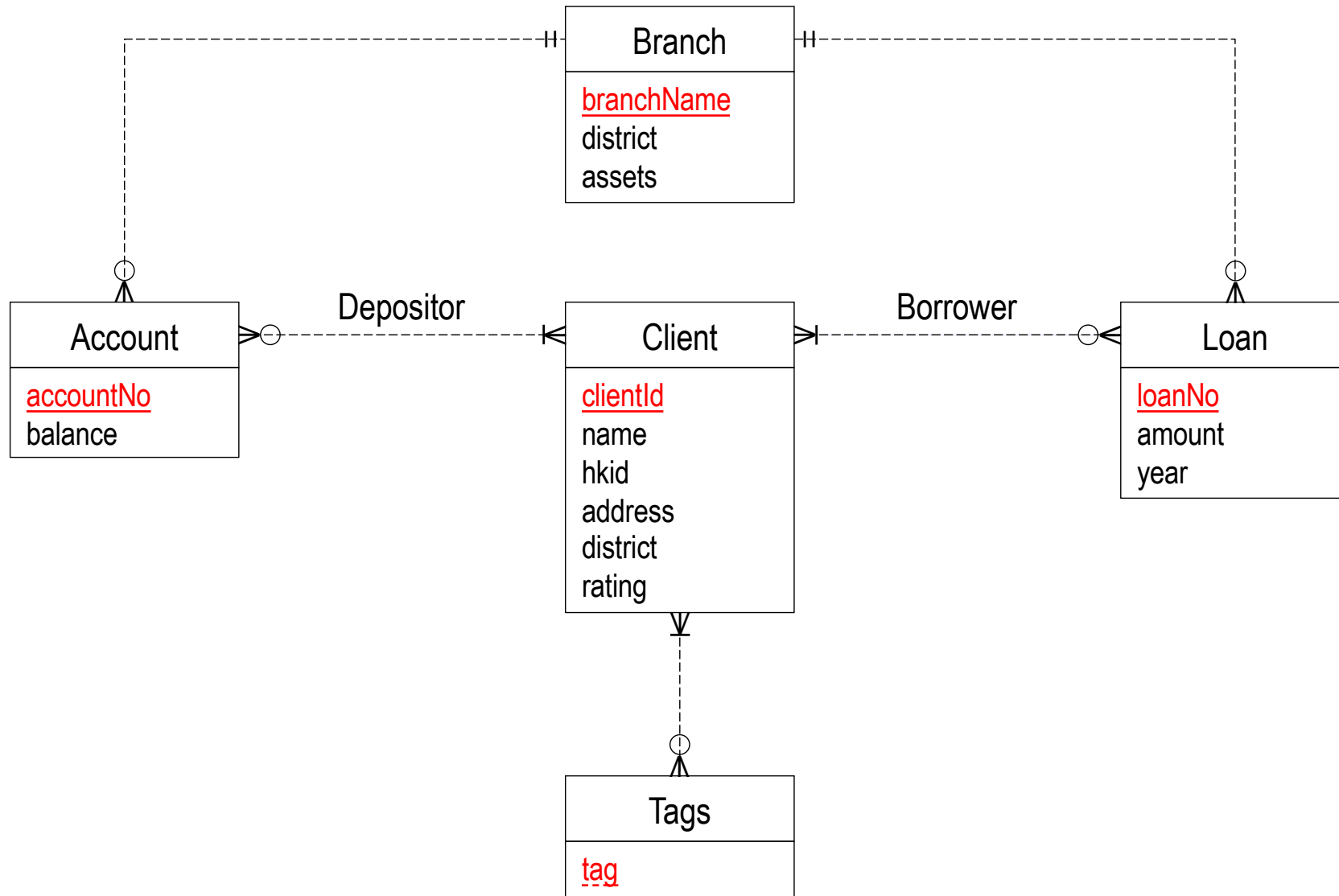
- **MongoDB** is the most popular open-source, schemaless NoSQL database system.
- It uses the concept of a document as its data model where a document is a **JSON**-like object (stored as **BSON**—**Binary JSON**).
- Besides the **JSON** data types, **MongoDB** also supports the data types:

Date	the JavaScript date class
BinData	binary data
Regular expression	regular expression
Object ID	the default type for the <code>_id</code> key of a document

DATA MODEL (CONT'D)

- A **document** is an **ordered set** of **key-value** pairs.
 - **Documents are like rows** in a relational database, but **each row** (document) can have a **different structure**.
 - A **key-value** pair is referred to as a **field**.
- A **collection** is a **set of documents**.
 - **Collections are like tables** in a relational database.
 - Each document in a collection has a **special field** (**key-value** pair) where the **key is named _id** and the **value must be unique within a collection**.
 - The value of the **_id** field can be user-defined or auto-generated by **MongoDB**.
- A **database** consists of **one or more collections**.

EXAMPLE BANK E-R SCHEMA



MONGODB DATABASE DESIGN

- A **MongoDB** database has a **flexible schema**, which means:
 - documents in a collection are **not** required to have the same set of fields.
 - a field's data type can differ between documents in a collection.
- In practice, documents in a collection **usually share a similar structure**.
- Document **validation rules** can be specified to **ensure consistency** in the schema.
- A key decision in designing a **MongoDB** database involves specifying **how relationships** between data **are represented**.
 - Relationships can be represented using either **references** or **embedded data**.

MONGODB DATABASE DESIGN: REFERENCES

- **References** represent relationships between data by **including 'links' from one document to another**.
 - Allows data to be normalized which may be more appropriate for representing many-to-many relationships or large hierarchical data.

Example: Include account references in client data or client references in account data or both.

clients collection

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road',  
  district: 'Central and Western', rating: 9,  
  accounts: ['A-117', 'A-215', 'A-301']},  
{clientId: 8, name: 'Isaac Li', hkid: 'K917263(6)', address: 'Man Lai Court', district: null,  
  rating: 6,  
  accounts: ['A-215']}
```

accounts collection

```
{accountNo: 'A-117', balance: 35000, branch: 'Star House', clients: [3]},  
{accountNo: 'A-215', balance: 80000, branch: 'Star House', clients: [3, 8]},  
{accountNo: 'A-301', balance: 75000, branch: 'Pacific Place', clients: [3]}
```

MONGODB DATABASE DESIGN: EMBEDDED DATA

- Embedded data represent relationships between data by storing related data in a single document in either a field or an array.
 - Allows related data to be retrieved or updated in a single operation.
 - Provides better performance for read operations.
 - May require data duplication since data is denormalized.

Example: Embed account data in client data.

clients collection

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road',  
  district: 'Central and Western', rating: 9,  
  accounts: [{accountNo: 'A-117', balance: 35000, branch: 'Star Housel'},  
             {accountNo: 'A-215', balance: 80000, branch: 'Star Housel'},  
             {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}}],  
  
{clientId: 8, name: 'Isaac Li', hkid: 'K917263(6)', address: 'Man Lai Court',  
  district: null, rating: 6,  
  accounts: [{accountNo: 'A-215', balance: 80000, branch: 'Star House'}]}
```

 **Whether to use references or embedded data will depend on an application's most common queries.**

BANK MONGODB DATABASE

clients collection

```
{clientId, name, hkid, address, district, rating,
  accounts: [{accountNo, balance, branch}],
  loans: [{loanNo, amount, year, branch}],
  tags: [ ]
}
```

```
{clientId: 3, name: 'Steven Chan', hkid: 'C835618(4)', address: 'Mainmount Road',
  district: 'Central and Western', rating: 9,
  accounts: [{accountNo: 'A-117', balance: 35000, branch: 'Star House'},
    {accountNo: 'A-120', balance: 80000, branch: 'Star House'},
    {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}],
  loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
    {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior', 'vip']}
```

```
{clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street',
  district: 'Islands', rating: 6,
  accounts: [{accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},
    {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}],
  loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
    {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}],
  tags: ['senior']}
```

```
{clientId: 8, name: 'Isaac Li', hkid: 'K917263(6)', address: 'Man Lai Court',
  district: null, rating: 6,
  accounts: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'}]}
```

```
{clientId: 17, name: 'Pam Du', hkid: 'W251984(6)', address: 'Tsing Yi Road',
  rating: 8,
  loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}],
  tags: ['vip']}
```

branches collection

```
{branch, district, liabilities, assets,
  accounts: [{accountNo, balance}],
  loans: [{loanNo, amount, year}]
}
```

```
{branch: 'Diamond Hill', district: 'Kwun Tong', liabilities: 55000,
  assets: 0,
  accounts: ['A-215']}
```

```
{branch: 'Festival Walk', district: 'Yau Tsim Mong', liabilities: 0,
  assets: 107000,
  loans: ['L-434', 'L-435']}
```

```
{branch: 'Pacific Place', district: 'Central and Western', liabilities: 120000,
  assets: 40000,
  accounts: ['A-301', 'A-322'],
  loans: ['L-321']}
```

```
{branch: 'Star House', district: 'Yau Tsim Mong', liabilities: 115000,
  assets: 115000,
  accounts: ['A-117', 'A-120'],
  loans: ['L-171']}
```



MONGODB QUERY LANGUAGE (MQL)

- **MongoDB Query Language (MQL)** is the interface by which clients interact with a MongoDB database.
- **MQL** commands can be executed
 - **interactively** using either the **MongoDB shell** (**mongosh**), a JavaScript interpreter, or **MongoDB Compass** a GUI that allows a database to be accessed using a desk-top-like environment.
 - **from within a programming language** using an appropriate driver.
- **MQL** supports basic **create**, **retrieve**, **update** and **delete** (**CRUD**) operations.
- **MQL** also supports an **aggregation framework**, based on the concept of a **data processing pipeline**, where documents enter a series of stages that filter and transform them.

AGGREGATION EXPRESSIONS

- **Aggregation expressions** are used to query and operate on documents and can be:
 - a **field path** with a **\$** prefix followed by the field's path in the document being accessed (e.g., '\$district' – the field path for the **district** field of clients, '\$accounts.balance' – the field path for the **balance** field of documents in the embedded **accounts** array).
 - an **operator** with a **\$** prefix (e.g., **\$cond**, **\$expr**, **\$count**, **\$sum**, **\$avg**, ... (mostly use in the aggregation framework—see later lecture).
 - a user or system **variable** with a **\$\$** prefix followed by a fixed name.
- These three categories of aggregation expressions **can be composed** when operating on input documents.

 **The dot notation is used to access the fields of embedded documents of a collection.**

 **When using the dot notation, the field name and embedded field name must be enclosed in (single or double) quotation marks.**

QUERYING DOCUMENTS

findOne(*queryFilter*, *projection*)

find(*queryFilter*, *projection*)

The **findOne** method returns **a single document** in a collection matching *queryFilter* (which specific document is returned is nondeterministic).

The **find** method returns **all documents** in a collection matching *queryFilter*.

The *queryFilter* parameter is a **JSON**-like document that specifies the selection criteria (i.e., like an **SQL where** clause). If *queryFilter* is empty or omitted, then either one document (**findOne**) is returned, or all documents (**find**) are returned.

Query: Find all client documents.

```
db.clients.find( );
```

≡

```
db.clients.find({ });
```

≡

```
db.clients.find({ },  
                { });
```

PROJECTION

The **find** method *projection* parameter is a **JSON**-like document that specifies which fields to **include in** or **exclude from** the query result.

- A value of **1** (or **true**) for a *projection* field means **include the field**; a value of **0** (or **false**) means **do not include the field**.
- The value of fields in the document that are absent, or **null**, are also included.
- By default, the **_id** field of a document is **always** included; to exclude it from a query result, specify **0** (or **false**) for the **_id** field.

Query: Find the districts of clients.

```
db.clients.find({ },  
  {district: 1});
```

≡

```
db.clients.find({ },  
  {district: true});
```

Query {**_id**: ObjectId('...'), **district**: 'Central and Western'}

result: {**_id**: ObjectId('...'), **district**: 'Islands'}
 {**_id**: ObjectId('...'), **district**: null}
 {**_id**: ObjectId('...')}

```
db.clients.find({ },  
  {_id: 0, district: 1});
```

≡

```
db.clients.find({ },  
  {_id: false, district: true});
```

Query result: {**district**: 'Central and Western'} {**district**: 'Islands'} {**district**: null} {}



PROJECTION (CONT'D)

- Inclusion and exclusion of fields cannot both be specified in a projection.

```
db.clients.find({ },  
  {district: 1, name: 0});
```

MQL Note

The inclusion of any field in a projection automatically excludes all other fields (except for the `_id` field).

 If **1** (inclusion) is specified for any field, then **0** (exclusion) cannot be specified for any other field, *except for the `_id` field*.

- Projection can exclude embedded data (objects and arrays).

Query: Find only the client information for clients.

```
db.clients.find({ },  
  {_id: 0, accounts: 0, loans: 0, tags: 0});
```

Query: Find only the loan information.

```
db.clients.find({ },  
  {_id: 0, loans: 1});
```

PROJECTION (CONT'D)

- The specified fields can be existing or newly computed fields.
- To **add a new field** or to **rename an existing field**, a new field name is specified, and its value set to some expression.

Query: Find the name of clients and their credit rating. Rename the field name as "Client name" and rating as "Creditworthiness".

```
db.clients.find({ },  
  {_id: 0, 'Client name': '$name', Creditworthiness: '$rating'});
```

Query {'Client name': 'Steven Chan', Creditworthiness: 9}

result: {'Client name': 'Mary Kwan', Creditworthiness : 6}

 {'Client name': 'Isaac Li', Creditworthiness : 6}

 {'Client name': 'Pam Du', Creditworthiness : 8}

MQL Note

Omitting the \$ when referencing fields that are being renamed causes the value in quotes to be interpreted as a literal string rather than as a field name.

SELECTION

- The **find** method *queryFilter* parameter specifies the criteria that documents must satisfy to be included in the query result.

Query: Find clients who live in Islands district.

```
db.clients.find({district: 'Islands'});
```

Query the complete document
result: for Mary Kwan

- The query result contains all documents, including their arrays *accounts*, *loans* and *tags*, where the district is Islands.
- To include only the client information, exclude the unwanted parts in the projection.

Query: Find only the client information for clients in Islands district.

```
db.clients.find({district: 'Islands'},  
  {_id: 0, accounts: 0, loans: 0, tags: 0});
```

Query result: {clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)', address: 'Hing Wah Street', district: 'Islands', rating: 6}

COMPARISON EXPRESSION OPERATORS

\$eq equal **\$ne** not equal **\$lt** less than **\$lte** less than or equal
\$gt greater than **\$gte** greater than or equal **\$cmp** compare two values

✎ **Since the default comparison operator is equality, it does not need to be specified.**

Query: Find only the client information for clients in Islands district.

```
db.clients.find({district: {$eq: 'Islands'}},  
  {_id: 0, accounts: 0, loans: 0, tags: 0});
```

This query produces the same result as the query on the previous slide.

Query {clientId: 4, name: 'Mary Kwan', hkid: 'M534109(3)',
result: address: 'Hing Wah Street', district: 'Islands', rating: 6}

Query: Find the names and districts of clients who do not live in Islands district.

```
db.clients.find({district: {$ne: 'Islands'}},  
  {_id: 0, name: 1, district: 1});
```

Query {name: 'Steven Chan', district: 'Central and Western'}
result: {name: 'Isaac Li', district: null}
 {name: 'Pam Du'}

✎ **By default, if a selection field is null or absent in a document, then the document is also in the query result.**

COMPARISON EXPRESSION OPERATORS: **\$CMP**

\$cmp: [*expression₁*, *expression₂*]

The **\$cmp** operator compares two values (both value and type) and returns:

- -1 if the first value is less than the second.
- 1 if the first value is greater than the second.
- 0 if the two values are equivalent.

Query: Compare the assets of branches to \$107,000.

```
db.branches.find({  
  {_id: 0, branch: 1, assets: 1, cmpTo107000: {$cmp: ['$assets', 107000]}});
```

Query {branch: 'Diamond Hill', assets: 0, cmpTo107000: -1}

result: {branch: 'Festival Walk', assets: 107000, cmpTo107000: 0}

{branch: 'Pacific Place', assets: 40000, cmpTo107000: -1}

{branch: 'Star House', assets: 115000, cmpTo107000: 1}

👉 **The \$cmp operator is primarily used with the aggregation framework.**

BOOLEAN EXPRESSION OPERATOR: \$AND

\$and: [{*predicate*₁}, {*predicate*₂}, ..., {*predicate*_n}]

The **\$and** operator performs a **logical and operation** on an **array of one or more predicates** and selects the documents that **satisfy all** the predicates.

👉 **The predicates are specified as an array of predicates.**

Query: Find the name of clients who live in Islands district whose credit rating is less than 8.

```
db.clients.find({$and: [{district: 'Islands', rating: {$lt: 8}},  
  {_id: 0, name: 1}]);
```

Query result:
{name: 'Mary Kwan'}

Since **and** is the default boolean operator, this query can also be expressed as:

```
db.clients.find({district: 'Islands', rating: {$lt: 8},  
  {_id: 0, name: 1});
```

Query result:
{name: 'Mary Kwan'}

👉 **These two queries get identical results *if and only if* the fields in all the predicates are different.**

BOOLEAN EXPRESSION OPERATOR: **\$AND** (CONTD)

Query: Find the name and rating of clients whose rating is between 7 and 9 (i.e., ≥ 7 and ≤ 9).

```
db.clients.find({$and: [{rating: {$gte: 7}, rating: {$lte: 9}}],  
  {_id: 0, name: 1, rating: 1});
```

Query result: {name: 'Steven Chan', rating: 9} {name: 'Pam Du', rating: 8}

```
db.clients.find({rating: {$gte: 7}, rating: {$lte: 9}},  
  {_id: 0, name: 1, rating: 1});
```

Query result: {name: 'Steven Chan', rating: 9} {name: 'Mary Kwan', rating: 6}
 {name: 'Isaac Li', rating: 6} {name: 'Pam Du', rating: 8}

 **These two queries get different results.**

For the second query, since both conditions are on the same field (rating), **the second condition overrides the first condition.**

Consequently, the query result includes all clients whose credit rating is less than or equal to 9 including those whose credit rating is less than 7.

BOOLEAN EXPRESSION OPERATOR: **\$AND** (CONTD)

Query: Find the name and rating of clients whose rating is between 7 and 9 (i.e., ≥ 7 and ≤ 9).

```
db.clients.find({rating: {$gte: 7, $lte: 9}},  
  {_id: 0, name: 1, rating: 1});
```

Query result: {name: 'Steven Chan', rating: 9} {name: 'Pam Du', rating: 8}

An **and** boolean **for the same field** can also be specified as an **array of conditions** as shown in the above query.

BOOLEAN EXPRESSION OPERATOR: \$OR

\$or: [{*predicate*₁}, {*predicate*₂}, ..., {*predicate*_n}]

The **\$or** operator performs a **logical or operation** on an **array** of one or more **predicates** and selects documents that **satisfy at least one** of the predicates.

👉 **The predicates are specified as an array of predicates.**

Query: Find the name and rating of clients whose rating is not between 8 and 10 (i.e., <8 or >10).

```
db.clients.find({$or: [{rating: {$lt: 8}}, {rating: {$gt: 10}}]},  
  {_id: 0, clientId: 1, rating: 1});
```

Query result: {name: 'Mary Kwan', rating: 6} {name: 'Isaac Li', rating: 6}

BOOLEAN EXPRESSION OPERATOR: \$OR (CONT'D)

Query: Find the name and rating of clients who live in Central and Western district or Islands district and whose credit rating is less than 8.

```
db.clients.find({$or: [{district: 'Central and Western'}, {district: 'Islands'}],  
               rating: {$lt: 8}},  
               {_id: 0, name: 1});
```

Query result: {name: 'Mary Kwan', rating: 6}

👉 **Clients whose rating field is null or absent are not included in the query result.**

```
db.clients.find({$and: [{ $or: [{district: 'Central and Western'}, {district: 'Islands'}]},  
                       rating: {$lt: 8}]},  
               {_id: 0, name: 1});
```

Query result: {name: 'Mary Kwan', rating: 6}

👉 **An and condition can be stated implicitly (as in the first query) or explicitly (as in the second query) with an or condition.**

BOOLEAN EXPRESSION OPERATOR: \$NOR

\$nor: [{*predicate*₁}, {*predicate*₂}, ..., {*predicate*_n}]

The **\$nor** operator performs a **logical *nor* operation** on an **array of one or more *predicates*** and selects documents that **fail to satisfy all the *predicates*** in the array.

👉 **The predicates are specified as an array of predicates.**

Query: Find the name of clients not in Central and Western or Islands districts.

```
db.clients.find({$nor: [{district: 'Central and Western'}, {district: 'Islands'}]},  
  {_id: 0, name: 1});
```

Query result: {name: 'Isaac Li'} {name: 'Pam Du'}

The query result will contain all documents that contain a district field whose value is **not equal** to Central and Western **or not equal** to Islands **or is null or** documents that do not contain the district field (i.e., the district field is absent).

BOOLEAN EXPRESSION OPERATOR: \$NOT

\$not: {*predicate*}

The **\$not** operator performs a **logical not operation** on a *predicate* and selects only those documents that **do not satisfy** the *predicate*.

Query: Find the name of clients not in Islands district.

```
db.clients.find({district: {$not: {$eq: 'Islands'}}},  
  {_id: 0, name: 1});
```

Query {name: 'Steven Chan', district: 'Central and Western'}

result: {name: 'Isaac Li', district: null}

{name: 'Pam Du'}

The query result will contain all documents that contain a district field and whose value is **not equal** to Islands **or** whose value is **null** **or** whose district field is absent.

The above query is equivalent to the following query.

```
db.clients.find({district: {$ne: 'Islands'}},  
  {_id: 0, name: 1});
```

NULL/ABSENT COMPARISON

- Specifying **null** for the value of a field returns documents where either the **field's value is null**, or **the field is absent**.

Query: Find the name of clients whose district field is null or is absent, i.e., the field does not exist in the document.

```
db.clients.find({district: null},  
  {_id: 0, name: 1});
```

Query result: {name: 'Isaac Li'} {name: 'Pam Du'}

- Specifying that embedded data (document or array) **should be null** (i.e., absent) will return all documents that have **no instance** of the specified embedded document or array.

Query: Find the name of clients who do not have a loan.

```
db.clients.find({loans: null},  
  {_id: 0, name: 1});
```

Query result: {name: 'Isaac Li'}



NULL/ABSENT COMPARISON (CONT'D)

- Specifying that embedded data (document or array) should not be null (i.e., should exist) will return all documents that have at least one instance of the specified embedded data (document or array).

Query: Find the name and loan numbers of the clients with loans.

```
db.clients.find({loans: {$ne: null}},  
  {_id: 0, name: 1, 'loans.loanNo': 1});
```

Query {name: 'Steven Chan', loans: [{loanNo: 'L-171'}, {loanNo: 'L-435'}]}
result: {name: 'Mary Kwan', loans: [{loanNo: 'L-321'}, {loanNo: 'L-434'}]}
 {name: 'Pam Du', loans: [{loanNo: 'L-435'}]}

NULL/ABSENT COMPARISON: **\$EXISTS**

*field: {**\$exists**: **boolean**}*

The **\$exists** operator matches documents that contain (**true**), or do not contain (**false**), a specified field, including documents where **the field is null** or **the field is absent**.

Query: Find the name and loan numbers of the clients with loans.

```
db.clients.find({loans: {$exists: true}},  
  {_id: 0, name: 1, 'loans.loanNo': 1});
```

Query result: the same as the query on the previous slide.

Query: Find the id and name of clients whose district field is absent.

```
db.clients.find({district: {$exists: false}},  
  {_id: 0, clientId: 1, name: 1});
```

Does not include clients whose
district field exists but is **null**.

Query result: {clientId: 17, name: 'Pam Du'}

 **The MQL **\$exists** operator does not correspond to the SQL exists operator.**

CONDITIONAL EXPRESSION OPERATOR: **\$IFNULL**

\$ifNull: [*expression*₁, ..., *expression*_n, *replacement-expression*]

The **\$ifNull** operator evaluates input expressions for null values and returns:

- The first non-null *expression* value found *or*
- The value of *replacement-expression* if all *expressions* evaluate to *null*.

➤ Undefined and absent fields are treated as null.

Query: Find the name and district of all clients; replace null or absent district values with 'unknown'.

```
db.clients.find({ },  
  { _id: 0, name: 1, district: { $ifNull: ['$district', 'unknown'] } });
```

Query {name: 'Steven Chan', district: 'Central and Western'}

result: {name: 'Mary Kwan', district: 'Islands'}

{name: 'Isaac Li', district: 'unknown'}

{name: 'Pam Du', district: 'unknown'}

👉 **Normally, only one input expression is specified.**

CONDITIONAL EXPRESSION OPERATOR: \$COND

\$cond: {if: *booleanExpression*, then: *trueCaseExpression*, else: *falseCaseExpression*}

\$cond: [*booleanExpression*, *trueCaseExpression*, *falseCaseExpression*]

The **\$cond** operator is an if-then-else statement that returns the value of the:

- *trueCaseExpression* if *booleanExpression* evaluates to **true**;
- *falseCaseExpression* if *booleanExpression* evaluates to **false**.
- Both the *trueCaseExpression* and the *falseCaseExpression* **must** be provided.
- If there is no action to take for the **else** (**false**) branch, then the **REMOVE** system variable can be used to specify that no action should be taken.

The **\$cond** operator can be used:

- in a projection to selectively project fields based on a condition.
- to specify different update actions based on a condition.
- to control the flow of data in an aggregation pipeline and apply different transformations based on a condition (see later lecture).

CONDITIONAL EXPRESSION OPERATOR: \$COND

(CONT'D)

Query: Find the name and rating of each client but only show the rating if it is less than or equal to 8.

```
db.clients.find({ },
  { _id: 0, name: 1, rating: { $cond: { if: { $lte: ['$rating', 8] },
    then: '$rating',
    else: '$$REMOVE' } } });
```

← \$\$REMOVE specifies that no action should be taken.

Query {name: 'Steven Chan'}
result: {name: 'Mary Kwan', rating: 6}
 {name: 'Isaac Li', rating: 6}
 {name: 'Pam Du', rating: 8}

👉 **Clients whose rating is null or absent are included in the query result.**

CONDITIONAL EXPRESSION OPERATOR: **\$SWITCH**

```
$switch: {  
  branches: [  
    {case: predicate, then: expression},  
    {case: predicate, then: expression},  
    .  
    .  
    .  
  ],  
  default: expression  
}
```

The **\$switch** operator evaluates a series of *predicates*. When it finds a *predicate* which evaluates to *true*, it executes a specified *expression* and breaks out of the control flow.

- *branches* is an array of control branch documents with fields *case* and *then*.
- *default* is optional and is taken if no branch case evaluates to *true*.

👉 **The case statements do not need to be mutually exclusive.**

CONDITIONAL EXPRESSION OPERATOR: \$SWITCH

(CONT'D)

Query: For each client categorize their credit rating as follows: less than 1, unknown risk; between 1 and 3, high risk; between 4 and 7, medium risk; between 8 and 10, low risk.

```
db.clients.find({ },
  { _id: 0, name: 1,
    rating: { $switch: {
      branches: [
        { case: // handle null and absent values
          { $lt: ['$rating', 1]}, then: 'unknown risk'},
        { case: { $lte: ['$rating', 3]}, then: 'high risk'},
        { case: { $lte: ['$rating', 7]}, then: 'medium risk'},
        { case: { $lte: ['$rating', 10]}, then: 'low risk'}
      ],
      default: 'error' }}}});
```

Query {name: 'Steven Chan', rating: 'low risk'}
result: {name: 'Mary Kwan', rating: 'medium risk'}
 {name: 'Isaac Li', rating: 'medium risk'}
 {name: 'Pam Du', rating: 'low risk'}

QUERY PREDICATE OPERATOR: **\$EXPR**

\$expr: {*expression*}

The **\$expr** operator allows the use of expressions within a query predicate and is required when

- comparing two fields from the **same document**.
- performing a calculation based on values from multiple fields in each document and then comparing the calculation to a constant.

Query: Find the name of the branches, their liabilities and their assets where their assets are less than their liabilities.

```
db.branches.find({$expr: {$lt: ['$assets', '$liabilities']}},  
  {_id: 0, branch: 1, liabilities: 1, assets: 1}  
);
```

Query {branch: 'Diamond Hill', liabilities: 55000, assets: 0}

result: {branch: 'Pacific Place', liabilities: 120000, assets: 40000}

👉 The **\$expr** operator requires that **aggregation expressions** (i.e., prefixing a field name with **\$**) be used to reference document fields.

REGULAR EXPRESSION OPERATOR: \$REGEX

field: {**\$regex**: */pattern/*, **\$options**: *options*}

The **\$regex** operator is used to match patterns in strings.

- The default pattern matching is case sensitive.
- *options* such as *i*, for case insensitive, can be specified.

Query: Find the name of clients whose name begins with Steven or Stephen (i.e., the name begins with 'Ste' followed by either 'v' or 'ph' followed by 'en' followed by any other characters).

```
db.clients.find({name: {$regex: /^Ste(v|ph)en/}},  
  {_id: 0, name: 1});
```

Query result:
{name: 'Steven Chan'}

Query: Find the name of clients with a double vowel (i.e., double a, e, i, o or u) in their name, regardless of case.

```
db.clients.find({name: {$regex: /[aeiou]{2}/, $options: 'i'},  
  {_id: 0, name: 1});
```

Query result:
{name: 'Isaac Li'}

EXAMPLE MQL QUERIES

Query: Find the id of clients who have a loan, an account, or both.

```
db.clients.find({$or: [{accounts: {$ne: null}}, {loans: {$ne: null}}]},  
  {_id: 0, clientId: 1});
```

```
db.clients.find({$or: [{accounts: {$exists: true}}, {loans: {$exists: true}}]},  
  {_id: 0, clientId: 1});
```

Query result: {clientId: 3} {clientId: 4} {clientId: 8} {clientId: 17}

Query: Find the id of clients who have both an account and a loan.

```
db.clients.find({accounts: {$ne: null}, loans: {$ne: null}},  
  {_id: 0, clientId: 1});
```

```
db.clients.find({accounts: {$exists: true}, loans: {$exists: true}},  
  {_id: 0, clientId: 1});
```

Query result: {clientId: 3} {clientId: 4}

EXAMPLE MQL QUERIES (CONT'D)

Query: Find the id of clients who have an account, but do not have a loan.

```
db.clients.find({accounts: {$ne: null}, loans: {$eq: null}},  
  {_id: 0, clientId: 1});
```

```
db.clients.find({accounts: {$exists: true}, loans: {$exists: false}},  
  {_id: 0, clientId: 1});
```

Query result: {clientId: 8}

Query: Find the client id, loan number, loan amount, year and branch name of the clients who have loans.

```
db.clients.find({loans: {$exists: true}},  
  {_id: 0, clientId: 1, loans: 1});
```

Query result: {clientId: 3: loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
 {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}
 {clientId: 4: loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
 {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}
 {clientId: 17: loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}